

ERP Lyon Copos — Documentação Técnica de Entrega

Relação do que está sendo entregue — repositório, documentação, scripts, banco e acessos — e de como o sistema opera. Complementa [SETUP.md](#) (instalação local), [ESTRUTURA_BANCO.md](#) (tabelas) e [INTEGRACOES.md](#) (APIs externas). Nenhuma senha ou chave está escrita aqui: os **nomes** das credenciais estão listados na seção 6 e os **valores** são entregues à CONTRATANTE por canal separado.

1. Arquitetura

navegador → Discloud (Node 18+, porta 8080) → Supabase (Postgres + Storage + Auth)
└─ backend/ Express – API em /api/*
└─ backend/public/ build do frontend (React 18 + Vite), servido pelo mesmo processo

Camada	Tecnologia	Onde
Frontend	React 18, Vite, TanStack Query, Tailwind (tema claro/escuro por classe)	frontend/ → build em backend/public/
Backend	Node.js, Express, @supabase/supabase-js , pg	backend/src/
Banco	Postgres (Supabase), projeto abtbkajjtuetactzsaou , região sa-east-1	Supabase
Arquivos	Supabase Storage — bucket público (imagens de catálogo) e privado (comprovantes, artes, fotos)	Supabase
Autenticação	Supabase Auth (e-mail + senha); perfil, setor e permissões em USUARIOS	Supabase
Hospedagem	Discloud, app lyoncopos.online , deploy automático a cada push na main	GitHub → Discloud
Domínio	lyoncopos.online (DNS apontado para o Discloud)	registrador do cliente

Multi-tenant por coluna: toda tabela de negócio tem [tenant_id](#); o middleware de autenticação resolve o tenant do usuário e todas as consultas filtram por ele.

2. Repositório e código-fonte

- **Repositório oficial (propriedade da Lyon Copos):** <https://github.com/coposlyon/ERP-Lyon> — branch [main](#).
- Cada commit descreve o que mudou e por quê (histórico completo desde o início do projeto). O [git log](#) é a trilha completa de alterações.
- Estrutura:

backend/ src/app.js src/config/env.js src/routes/*.js src/lib/atencao.js src/lib/fluxoPedido.js src/lib/pedidoAutomacao.js src/lib/prazoProducao.js scripts/migrate.js public/ frontend/ src/pages/<Modulo>/ src/components/Fluxo/ migrations/NNN_*.sql docs/	servidor, migrações automáticas no boot carrega .env e valida variáveis uma rota por módulo (sales, financial, production, contador...) catálogo dos 28 status, módulos e fases regras de quem move o quê, requisitos por etapa avanços automáticos (pagamento, arte, estoque) data-limite de saída em dias úteis + alerta 24h aplica migrações; google-token.js; seeds de teste build do frontend (commitado) telas; src/lib/menu.js é o registro de telas e permissões fila de etapas genérica (Designer, Produção, Logística) esquema do banco, em ordem, com comentário do porquê este documento, manual e treinamento
---	--

3. Deploy e operação

Deploy: `git push origin main` → o Discloud reconstrói e reinicia. O servidor **precisa** ouvir na porta **8080** no Discloud (forçado no código quando detecta o ambiente). Variáveis de ambiente novas só valem após um **novo deploy** (não bastam reiniciar).

Build do frontend: `cd frontend && npm run build` gera `backend/public/`, que é commitado — o Discloud não roda o build.

Migrações: em `migrations/`, numeradas. Rodam sozinhas no boot (`AUTO_MIGRATE`) ou à mão com `node backend/scripts/migrate.js --dry` só lista). O runner grava em `_MIGRATIONS` e avisa se dois arquivos tiverem o mesmo número. Estado atual: **122 migrações aplicadas**.

Logs: painel do Discloud (stdout). Erros de rota aparecem como `[modulo/rota] mensagem`.

Conexão ao banco fora do app: usar o **pooler** do Supabase (`aws-1-sa-east-1.pooler.supabase.com:5432`, usuário `postgres.<projeto>`); o host direto `db.<projeto>.supabase.co` é só IPv6 e não responde do Discloud.

4. Segurança da informação

Exigência	Como está atendida
Backups redundantes	Supabase: backups diários automáticos do Postgres (plano do projeto) + PITR quando habilitado. Recomenda-se ainda um <code>pg_dump</code> mensal guardado fora do Supabase (comando na seção 5).
Trilha de auditoria	Tabela <code>AUDITORIA</code> (quem, o quê, quando, em qual registro, detalhes) alimentada por todas as rotas de escrita; <code>production_log</code> em cada pedido com cada status, hora e usuário; histórico de cada conta a receber. Nada é apagado — só cancelado/desativado.
Controle de acessos	Supabase Auth + setor (<code>Configurações → Permissões por setor</code>) + permissão por tela no cadastro do colaborador; o servidor valida o módulo em cada rota (<code>requireModules</code>). Senha dupla para avançar etapas na fábrica. Chaves de serviço só no servidor.
Recuperação de desastre	Código no GitHub; banco restaurável pelo painel do Supabase (backup diário) ou por <code>pg_restore</code> do dump; app recriável no Discloud em minutos a partir do repositório + variáveis (seção 6). Procedimento na seção 5.
LGPD	Dados pessoais só nas tabelas de cadastro; portal do cliente exige CPF + dado pessoal para abrir o pedido; comprovantes e artes em bucket privado com link temporário; <code>.env</code> fora do repositório (<code>.gitignore</code>).

5. Procedimentos

Backup manual do banco

```
pg_dump "$DATABASE_URL" --no-owner --format=custom --file=lyon_$(date +%Y%m%d).dump
```

Restaurar (em um banco vazio ou novo projeto Supabase)

```
pg_restore --no-owner --dbname="$DATABASE_URL" lyon_YYYYMMDD.dump
```

Recriar o app do zero (disaster recovery)

1. Discloud → novo app tipo *site*, conectar ao GitHub `coposlyon/ERP-Lyon`, branch `main`.
2. Colar as variáveis da seção 6 (valores entregues à parte).
3. Deploy. O boot aplica migrações pendentes e sobe na porta 8080.
4. Apontar o DNS do domínio para o Discloud (se mudou).

Dar acesso a um novo desenvolvedor

1. GitHub: adicionar como colaborador em [coposlyon/ERP-Lyon](#).
2. Supabase: convidar para a organização com papel *Developer*.
3. Discloud: adicionar como membro do app (ou compartilhar o painel).
4. Ele roda [SETUP.md](#) localmente com um [.env](#) próprio.

Rotina mensal recomendada: ver *Manual* → 12 e *Treinamento* → *Encontro 6* (fechamento). Tecnicamente: conferir o painel do Supabase (uso de banco/storage), baixar um dump, e verificar em [Contábil](#) → [Alertas](#) a validade do certificado digital.

6. Credenciais e acessos a entregar (nomes)

Entregues à Lyon Copos em canal separado, nunca neste arquivo nem no repositório.

Contas administrativas

- GitHub — organização/usuário [coposlyon](#) (dono do repositório)
- Supabase — organização e projeto [abtbkajjtuetactzsaou](#) (dono)
- Discloud — conta dona do app [lyoncopos.online](#)
- Registrador do domínio [lyoncopos.online](#)
- Google Cloud — projeto do OAuth "lyoncoposwebcont" (Google Contatos)
- Usuário **Administrador** do ERP

Variáveis de ambiente do servidor (as 11 em produção hoje; o [.env.example](#) documenta todas as ~60 opcionais)

[NODE_ENV](#) [PORT](#) [FRONTEND_URL](#) [SUPABASE_URL](#) [SUPABASE_ANON_KEY](#) [SUPABASE_SERVICE_KEY](#) [DATABASE_URL](#) [GROQ_API_KEY](#)
[GOOGLE_CLIENT_ID](#) [GOOGLE_CLIENT_SECRET](#) [GOOGLE_REFRESH_TOKEN](#)

Integrações que dependem de credenciais da própria Lyon (código pronto; ativar é colar as variáveis e fazer um deploy)

- Certificado digital A1 + emissor de NF-e ([fiscal](#))
- Banco: boleto/CNAB ([BOLETO_*](#)), PIX com gateway ([MP_*](#)) — hoje PIX manual Nubank por decisão do cliente
- Transportadoras: Total Express ([TOTALEXPRESS_*](#), bloqueado por liberação de IP do lado deles), Braspress, Melhor Envio
- WhatsApp Business ([WHATSAPP_*](#)), Meta/Instagram ([FB_*](#), [IG_*](#)), e-mail ([RESEND_*](#))
- Leiaute do escritório contábil para a exportação (hoje genérica em Excel/CSV)

Com a entrega dos acessos acima, a Lyon Copos passa a ter controle integral do sistema: código, banco, hospedagem, domínio e integrações. Qualquer desenvolvedor consegue operar e evoluir o ERP a partir deste documento, do [SETUP.md](#) e do histórico do repositório.